Лабораторна робота №16

Тема: Валідатор реєстраційної форми.

Мета: Формування навичок перевірки даних введених користувачем у Javascript.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Введення в браузерні події

Подія це сигнал від браузера про те, що щось сталося. Усі DOM-вузли подають такі сигнали (хоча події бувають і не тільки в DOM).

Ось список найбільш часто використовуваних DOM-подій, поки просто для ознайомлення:

Події миші:

- `click` - відбувається, коли скликали на елемент лівою кнопкою миші (на пристроях з сенсорними екранами воно відбувається при торканні).
- `contextmenu` - відбувається, коли скликали на елемент правою кнопкою миші.
- `mouseover/mouseout` – коли миша наводиться на / покидає елемент.
- `mousedown/mouseup` - коли натиснули / відпустили кнопку миші на елементі.
- `mousemove` - при переміщеннні миші.

Події на елементах управління:

- `submit` - користувач відправив форму `<form>`.
- `focus` - користувач фокусується на елементі, наприклад натискає на `<input>`.

Клавіатурні події:

- `keydown i keyup` - коли користувач натискає / відпускає клавішу.

Події документа:

- `DOMContentLoaded` - коли HTML завантажений і оброблений, DOM документа повністю побудований і доступний.

CSS events:

- `transitionend` - коли CSS-анімація завершена.

Існує безліч інших подій. Ми детально розберемо їх в наступних розділах.

Обробники подій

Події можна призначити *обробник* , тобто функцію, яка спрацює, як тільки подія відбулася.

Саме завдяки обробникам JavaScript-код може реагувати на дії користувача.

Є кілька способів призначити події обробник. Зараз ми їх розглянемо, починаючи з найпростішого.

Використання атрибута HTML

Оброблювач може бути призначений прямо в розмітці, в атрибуті, який називається `on<event>`.

Наприклад, щоб призначити обробник події `click` на елементі `input`, можна використовувати атрибут `onclick`, ось так:

```
<input value="Нажми мене" onclick="alert('Клік!')"
type="button">
```

При натисканні мишкою на кнопці виконається код, вказаний в атрибуті `onclick`.

Зверніть увагу, для вмісту атрибута `onclick` використовуються одинарні лапки, так як сам атрибут знаходиться в подвійних. Якщо ми забудемо про це і поставимо подвійні лапки всередині атрибута, ось так: `onclick="alert("Click!")"`, код не буде працювати.

Атрибут HTML-тега - не найзручніше місце для написання великої кількості коду, тому краще створити окрему JavaScript-функцію і викликати її там.

Наступний приклад при натисканні запускає функцію `countRabbits()`:

```
  <script>
    function countRabbits() {
      for(let i=1; i<=3; i++) {
        alert("Кролик номер " + i);
      }
    }
  </script>

  <input type="button" onclick="countRabbits()"
value="Рахувати кроликів!">
```

Використання властивості DOM-об'єкта

Можна призначати обробник, використовуючи властивість DOM-елемента `on<подія>`.

Наприклад, `elem.onclick`:

```
  <input id="elem" type="button" value="Нажми мене!">
  <script>
    elem.onclick = function() {
      alert('Спасибі');
    };
  </script>
```

Якщо обробник заданий через атрибут, то браузер читає HTML-розмітку, створює нову функцію з вмісту атрибута і записує в властивість.

Цей спосіб, по суті, аналогічний попередньому.

Оброблювач завжди зберігається у властивості DOM-об'єкта, а атрибут - лише один із способів його ініціалізації.

Ці два приклади коду працюють однаково:

Тільки HTML:

```
<input type="button" onclick="alert('Клік!')" value="Кнопка">
```

HTML + JS:

```
<input type="button" id="button" value="Кнопка">
<script>
  button.onclick = function() {
    alert('Клік!');  };
```

```
</script>
```

Так як у елемента DOM може бути тільки одна властивість з ім'ям onclick, то призначити більше одного обробника так не можна.

У прикладі нижче призначення через JavaScript перезапише обробник з атрибута:

```
<input type="button" id="elem" onclick="alert('Було')"
value="Нажми мене">
<script>
  elem.onclick = function() { // перезапише обробник
    alert('Стане'); // виділяє тільки це
  };
</script>
```

До речі, оброблювачем можна призначити і вже існуючу функцію:

```
function sayThanks() {
  alert('Спасибі!');
}
elem.onclick = sayThanks;
```

Прибрати обробник можна призначенням `elem.onclick = null`.

Доступ до елементу через this

Усередині обробника події `this` посилається на поточний елемент, тобто на той, на якому, як кажуть, «висить» (тобто призначений) обробник.

У коді нижче `button` виводить свій вміст, використовуючи `this.innerHTML`:

```
<button onclick="alert(this.innerHTML)">Нажми меня</button>
```

Використовуйте саме функції, а не рядки.

Призначення обробника рядком `elem.onclick = "alert(1)"` також спрацює. Це зроблено з міркувань сумісності, але робити так не рекомендується.

addEventListener

Фундаментальний недолік описаних вище способів призначення обробника - неможливість повісити кілька обробників на одну подію.

Наприклад, одна частина коду хоче при кліці на кнопку робити її підсвічується, а інша - видавати повідомлення.

Ми хочемо призначити два обробника для цього. Але нове DOM-властивість перезапише попереднє:

```
input.onclick = function() { alert(1); }
// ...
input.onclick = function() { alert(2); } // зміна
попереднього обробника
```

Розробники стандартів досить давно це зрозуміли і запропонували альтернативний спосіб призначення обробників за допомогою спеціальних методів `addEventListener` і `removeEventListener`.

Синтаксис додавання обробника:

```
element.addEventListener(event, handler[, options]);
```

`event`

Ім'я події, наприклад `"click"`.

`handler`

Посилання на функцію-обробник.

`options`

Додатковий об'єкт з властивостями:

- `once`: Якщо `true`, тоді обробник буде автоматично видалений після виконання.
- `capture`: Фаза, на якій повинен спрацювати обробник. Так історично склалося, що `options` може бути `false/true`, це те ж саме, що `{capture: false/true}`.
- `passive`: Якщо `true`, то вказує, що обробник ніколи не викличе `preventDefault()`.

Для видалення обробника слід використовувати `removeEventListener`:

```
element.removeEventListener(event, handler[, options]);
```

Видалення вимагає саме ту ж функцію

Для видалення потрібно передати саме ту функцію-обробник яка була призначена.

```
function handler() {
  alert( 'Спасибі!' );
}
input.addEventListener("click", handler);
// ....
input.removeEventListener("click", handler);
```

Звернемо увагу - якщо функцію обробник не збереглися де-небудь, ми не зможемо її видалити. Немає методу, який дозволяє отримати з елемента обробники подій, призначені через `addEventListener`.

Метод `addEventListener` дозволяє додавати кілька обробників на одну подію одного елемента, наприклад:

```
<input id="elem" type="button" value="Нажми мене"/>
<script>
  function handler1() {
    alert('Спасибі!');
  };
  function handler2() {
    alert('Спасибі ще раз!');
  }
  elem.onclick = () => alert("Привіт");
  elem.addEventListener("click", handler1); // Спасибі!
  elem.addEventListener("click", handler2); // Спасибі ще
раз!
</script>
```

Як видно з прикладу вище, можна одночасно призначати обробники і через DOM-властивість і через `addEventListener`. Однак, щоб уникнути плутанини, рекомендується вибрати один спосіб.

Обробники деяких подій можна призначати тільки через addEventListener

Існують події, які не можна призначити через DOM-властивість, але можна через `addEventListener`.

Наприклад, таке подія `DOMContentLoaded`, яке спрацьовує, коли завершено завантаження і побудова DOM документа.

```
document.onDOMContentLoaded = function() {
  alert("DOM побудований"); // не працює
};
document.addEventListener("DOMContentLoaded", function() {
  alert("DOM побудований "); // а так працює
});
```

Так що `addEventListener` більш універсальний. Хоча зауважимо, що таких подій меншість, це швидше виняток, ніж правило.

Об'єкт події

Щоб добре обробити подія, можуть знадобитися деталі того, що сталося. Не просто «клік» або «натискання клавіші», а також - які координати покажчика миші, яка клавіша натиснута і так далі.

Коли відбувається подія, браузер створює *об'єкт події* , записує в нього деталі і передає його в якості аргументу функції-обробника.

Приклад нижче демонструє отримання координат миші з об'єкта події:

```
<input type="button" value="Нажми меня" id="elem">

<script>
  elem.onclick = function(event) {

    alert(event.type + " на " + event.currentTarget);
    alert("Координати: " + event.clientX + ":" +
event.clientY);
  };
</script>
```

Деякі властивості об'єкта event:

`event.type` Тип події, в даному випадку `"click"`.

`event.currentTarget` Елемент, на якому спрацював обробник. Значення - зазвичай таке ж, як і у `this`, але якщо обробник є функцією-стрілкою або за допомогою `bind` прив'язаний інший об'єкт в якості `this`, то ми можемо отримати елемент з `event.currentTarget`.

`event.clientX / event.clientY`

Координати курсора в момент кліка щодо вікна, для подій миші.

Є також і ряд інших властивостей, в залежності від типу подій, які ми розберемо в подальших главах.

Об'єкт події доступний і в HTML

При призначенні обробника в HTML, теж можна використовувати об'єкт `event`, ось так:

```
<input type="button" onclick="alert(event.type)" value="Тип події">
```

Це можливо тому, що коли браузер з атрибута створює функцію-обробник, то вона виглядає так: `function(event) { alert(event.type) }`. Тобто, її перший аргумент називається `"event"`, а тіло взято з атрибута.

ОТЖЕ

Є три способи призначення обробників подій:

1. Атрибут HTML: **`onclick="..."`**.
2. DOM-властивість: **`elem.onclick = function`**.
3. Спеціальні методи: **`elem.addEventListener`**`(event, handler[, phase])` для додавання, `removeEventListener` для видалення.

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

<div class="container mt-5">
  <div class="card shadow-sm mx-auto" style="max-width: 500px;">
    <div class="card-header bg-primary text-white">
      <h5 class="mb-0">Внесення реквізитів статті</h5>
    </div>
    <div class="card-body">
      <form id="article-form" novalidate>

        <div class="mb-3">
          <label for="author" class="form-label">Автор (Прізвище І.О.) *</label>
          <input type="text" class="form-control" id="author" placeholder="Качурівський В.О.">
          <div class="invalid-feedback">Формат: Прізвище І. (або І.П.) українською чи латиницею.</div>
        </div>

        <div class="mb-3">
          <label for="year" class="form-label">Рік видання *</label>
          <input type="text" class="form-control" id="year" placeholder="2026">
          <div class="form-text">Тільки 4 цифри (напр. 2025).</div>
        </div>

        <div class="mb-3">
          <label class="form-label">Сторінки публікації (з - по) *</label>
          <div class="input-group">
```

```
                    <input type="number" class="form-control" id="page-from" placeholder="З">
                    <span class="input-group-text">-</span>
                    <input type="number" class="form-control" id="page-to" placeholder="По">
                    <div class="invalid-feedback">Кінцева сторінка має бути більшою за початкову.</div>
                  </div>
                </div>

                <div class="d-grid gap-2 d-md-flex justify-content-md-end mt-4">
                  <button type="reset" class="btn btn-outline-secondary btn-sm">Очистити</button>
                  <button type="submit" class="btn btn-primary">Далі →</button>
                </div>
              </form>
            </div>
          </div>
        </div>
```

1. Текстова валідація за шаблоном (Подія `blur`)

Реалізуйте перевірку поля **«Автор ЗВО»**.

- **Правило:** Дані мають відповідати формату `Прізвище І.О.` (наприклад, `Качурівський В.О.`).
- **Дія:** Якщо формат невірний, змініть колір рамки поля на червоний. Якщо вірний — на зелений.

2. Числова валідація (Подія `input`)

Реалізуйте «живу» перевірку поля **«Рік видання»**.

- **Правило:** Користувач може вводити лише цифри. Довжина не повинна перевищувати 4 символи.
- **Дія:** Блокуйте введення будь-яких символів, крім цифр. Якщо рік > 2026, виводьте попередження поруч із полем.

3. Логічна валідація діапазону (Подія `change`)

Реалізуйте перевірку **сторінок публікації** (поля «з» та «по»).

- **Правило:** Число в полі «по» не може бути меншим за число в полі «з».
- **Дія:** При зміні значень, якщо правило порушено, виведіть `alert` з повідомленням: *"Кінцева сторінка не може бути меншою за початкову"*.

4. Фінальна перевірка (Подія `click` на кнопці «Далі»)

- **Дія:** Перевірте, чи заповнене поле **«Повна назва наукової статті»**. Якщо воно порожнє — зупиніть перехід до наступного кроку та підсвітіть поле.